

TYPES AND MUTABILITY

Type	Mutable	Hashable	Note
int float bool str bytes	no	yes	bool is an int
tuple	no	if items are	fixed record
frozenset	no	yes	immutable set
list	yes	no	ordered, dynamic array
dict	yes	no	insertion-ordered (3.7+)
set	yes	no	unique, unordered

- Dict keys and set members must be hashable. `None` is a singleton: test with `is None`.
- Truthiness: `0`, `0.0`, `''`, `[]`, `{}`, `set()`, `None` are falsy. `[] == False` is `False`; `bool([])` is `False`.
- `==` compares values, `is` compares identity. Small ints and interned strings may share objects: never rely on `is` for values.

NUMBERS, TRUTHINESS, FORMATTING

<code>7 // 2, -7 // 2, -7 % 2</code>	<code># 3, -4, 1 (floors)</code>
<code>0.1 + 0.2 == 0.3</code>	<code># False</code>
<code>math.isclose(0.1 + 0.2, 0.3)</code>	<code># True</code>
<code>int("3.7")</code>	<code># ValueError</code>
<code>int(float("3.7")), int(3.99)</code>	<code># 3, 3 (truncates)</code>
<code>bool("False"), bool([0])</code>	<code># True, True</code>
<code>f"{0.4567:.1%} {42:05d}"</code>	<code># '45.7% 00042'</code>
<code>f"{'a':<4} {'b':>4} {n:}"</code>	<code># align; debug</code>

- Compare floats with a tolerance. `round(2.5)` is `2`: ties go to even.
- Text from a file is truthy even when it says "False": compare `s.lower() == "true"`. Use `x is None`, not `x or d`, when `0` is valid.

COMPLEXITY OF BUILT-INS

Operation	Cost
<code>list[i]</code> , <code>append</code> , <code>pop()</code>	$O(1)$
<code>list.insert(0, x)</code> , <code>pop(0)</code> , <code>x in list</code>	$O(n)$
<code>dict/set</code> <code>get</code> , <code>set</code> , <code>in</code> , <code>delete</code>	$O(1)$ avg
<code>sorted</code> , <code>list.sort</code>	$O(n \log n)$, stable
<code>deque</code> <code>append/pop</code> both ends	$O(1)$
<code>heapq</code> <code>push / pop</code> smallest	$O(\log n)$
<code>bisect</code> <code>search</code> in sorted list	$O(\log n)$
<code>s += t</code> in a loop	$O(n^2)$: use <code>''.join(parts)</code>
<code>slice a[i:j]</code>	$O(j - i)$ copy

- Amortised $O(1)$ `append`: the array over-allocates and doubles.

SYNTAX YOU MUST KNOW COLD

<code>a, *mid, z = [1, 2, 3, 4]</code>	<code># unpacking</code>
<code>a, b = b, a</code>	<code># swap</code>
<code>x[::-1]; x[1:-1]; x[::2]</code>	<code># slices</code>
<code>[f(i) for i in r if i % 2]</code>	<code># list comp</code>
<code>{k: v for k, v in pairs}</code>	<code># dict comp</code>
<code>(i * i for i in r)</code>	<code># generator</code>
<code>for i, v in enumerate(xs, 1): ...</code>	
<code>for a, b in zip(xs, ys): ...</code>	<code># shortest</code>
<code>sorted(xs, key=lambda t: (-t[1], t[0]))</code>	
<code>max(xs, key=len); any(...); all(...)</code>	
<code>y = a if cond else b</code>	
<code>if (n := len(xs)) > 3: ...</code>	<code># walrus</code>
<code>f"{x:.2f} {n:,} {s!r:>8}"</code>	<code># f-string</code>

- `sorted` returns a new list; `.sort()` sorts in place and returns `None`.
- `dict.get(k, d)`, `setdefault`, `d | e` merge (3.9+), `{**a, **b}`.

METHODS WORTH REMEMBERING

- list**: `append` `extend` `insert` `remove` `pop` `index` `count` `sort` `reverse` `clear`; `xs + ys` copies; `xs * 3` repeats.
- dict**: `keys` `values` `items` `get` `pop` `popitem` `update` `setdefault`; iterate for `k, v in d.items()`.
- set**: `add` `discard` `remove` `union` `|` `intersection` & `difference` - `symmetric_difference` `^`, `issubset`; `x in s` is $O(1)$.
- Numbers**: `divmod(a, b)`, `round(x, 2)` (banker's rounding), `abs` `pow` `sum` `min` `max`, `int("12")`, `float`, `math.sqrt` `floor` `ceil` `log` `gcd` `inf`.
- `range(a, b, step)` is lazy; `list(range(5)).reversed`, `sorted(reverse=True)`, `zip(*matrix)` transposes.

- Chained comparison: `0 < x < 10`. and/or return an operand: `x or default`.

FUNCTIONS AND SCOPE

```
def f(a, b=2, *args, c, **kw): ...
def f(x, /, y, *, z): ...      # pos-only / kw-only
lambda x: x * 2
nonlocal n                    # rebind enclosing variable
```

- LEGB** lookup: Local, Enclosing, Global, Built-in.
- Arguments are passed by object reference: mutating a passed list changes the caller's list; rebinding the name does not.

TRAP Mutable default argument: `def f(x, acc=[])` shares one list across calls. Use `acc=None` then `acc = acc or []`. **Late binding closures:** `[lambda: i for i in range(3)]` all return `2`. Bind with `lambda i=i: i`.

- Decorator = function that takes a function and returns a function; use `functools.wraps`.
- Generator = function with `yield`; lazy, one pass, constant memory. `yield` from delegates.

LOOPS AND ITERATORS

```
for x in xs:
    if ok(x): break
else: ...
it = iter(xs); next(it, None) # default: no error
next(iter(rows), None)      # peek at first item
list(islice(gen(), 10))     # first 10 of a stream
zip(a, b, strict=True)      # error if lengths differ
ks, vs = zip(*pairs)        # unzip
for blk in iter(lambda: f.read(4096), b''):
```

- A `for` loop calls `next()` until `StopIteration`. A generator is spent after one pass: `list(g)` twice gives `[]` the second time.
- Laziness saves memory, not time. Batch pattern: fill a list to size `n`, `yield` it, `reset`, then `yield` the leftover.

COPYING AND EQUALITY

- Assignment never copies. `b = a` is the same object.
- Shallow copy (`a.copy()`, `a[:]`, `list(a)`, `copy.copy`) copies the container, shares the items. `copy.deepcopy` copies recursively.
- `[[0] * 3] * 3` makes three references to one row. Use `[[0] * 3 for _ in range(3)]`.
- Strings are immutable; `s[0] = 'a'` raises `TypeError`.

OOP AND SPECIAL METHODS

- `__init__` initialises; `__new__` creates. `__repr__` for developers (unambiguous), `__str__` for users.
- Operators: `__eq__` `__lt__` `__hash__` `__len__` `__getitem__` `__iter__` `__next__` `__contains__` `__call__` `__enter__` `__exit__`.
- Defining `__eq__` without `__hash__` makes the class unhashable.
- @property computed attribute; @classmethod gets `cls` (alt constructors); @staticmethod gets nothing.
- Inheritance: `super().__init__()`; method resolution order via `C3` (`Cls.__mro__`).
- @dataclass generates `__init__`, `__repr__`, `__eq__`; `frozen=True` for immutable. `__slots__` saves memory.
- Duck typing: "if it quacks"; use ABCs or `Protocol` for contracts.

ERRORS, CONTEXT MANAGERS

```
try:
    risky()
except (KeyError, ValueError) as e:
    handle(e)
else:
    ok() # no exception
finally:
    cleanup() # always
with open(p) as f: ... # closes on exit
raise ValueError("msg") from e
```

Catch specific exceptions, never bare `except:`. Custom errors: subclass `Exception`.

COLLECTIONS

```

from collections import (Counter, defaultdict,
                        deque, namedtuple)

Counter("aab").most_common(1)      # [('a', 2)]
d = defaultdict(list); d[k].append(v)
q = deque(maxlen=3); q.appendleft(x)
q.popleft()                        # 0(1) queue
P = namedtuple("P", "x y")

```

ITERTOOLS, FUNCTOOLS, HEAPQ, BISECT

```

from itertools import (permutations, combinations,
                      product, accumulate, groupby, chain, islice)
list(combinations("abc", 2))       # ab ac bc
list(accumulate([1, 2, 3]))       # [1, 3, 6]
import heapq
h = []; heapq.heappush(h, x); heapq.heappop(h)
heapq.nlargest(3, xs)              # top-k
heapq.heappush(h, -x)              # max-heap trick
import bisect; bisect.bisect_left(xs, x)
from functools import lru_cache, partial, reduce
@lru_cache(maxsize=None)          # memoise

```

groupby needs sorted input (it groups neighbours only). heapq is a min-heap only. chain(a, b) joins iterables; product(a, b) gives every pairing (a parameter grid); islice slices any iterator.

STRINGS

- split(), ' '.join(xs), strip lstrip rstrip, lower upper title, startswith endswith, find (-1) vs index (error), replace, count, isalpha isdigit isalnum, zfill, partition.
- ord('a') = 97, chr(97). Reverse s[::-1]. Palindrome: s == s[::-1].
- Anagram: sorted(a) == sorted(b) or Counter(a) == Counter(b).
- Regex: re.findall(r"\d+", s), re.sub(r"\s+", " ", s), re.match anchors at start, re.search anywhere; groups (...), lazy .*?.

CLASSIC SNIPPETS

```

# two-sum in O(n)
seen = {}
for i, x in enumerate(nums):
    if t - x in seen: return seen[t - x], i
    seen[x] = i
# dedupe, keep order
list(dict.fromkeys(xs))
# first non-repeating char
c = Counter(s)
next((ch for ch in s if c[ch] == 1), None)
# max subarray (Kadane)
best = cur = a[0]
for x in a[1:]:
    cur = max(x, cur + x); best = max(best, cur)
# flatten one level
[y for x in xs for y in x]
# word frequency
Counter(text.lower().split())

```

FROM SCRATCH: STATS AND ML

```

import math
def mean(x): return sum(x) / len(x)
def var(x, ddof=1):
    m = mean(x)
    return sum((v - m) ** 2 for v in x) / (len(x) - ddof)
def std(x): return math.sqrt(var(x))
def sigmoid(z): return 1 / (1 + math.exp(-z))
def softmax(z):
    m = max(z); e = [math.exp(v - m) for v in z]
    return [v / sum(e) for v in e]
def dist(a, b):
    return math.sqrt(sum((p - q) ** 2 for p, q in zip(a, b)))

```

Sample variance divides by n - 1. Subtract the max in softmax for numerical stability.

FROM SCRATCH: GRADIENT DESCENT

```

# fit y = w*x + b by gradient descent
w = b = 0.0; lr = 0.01
for _ in range(1000):
    gw = gb = 0.0
    for x, y in data:
        err = (w * x + b) - y
        gw += 2 * err * x / len(data)
        gb += 2 * err / len(data)
    w -= lr * gw; b -= lr * gb

```

K-means: assign each point to the nearest centre, move each centre to its group's mean, repeat. KNN: sort by distance, majority vote of the k nearest.

FILES, JSON, DATES

```

from pathlib import Path; import json
txt = Path("a.txt").read_text()
data = json.loads(s); json.dumps(d, indent=2)
from datetime import datetime, timedelta
datetime.strptime("2025-01-31", "%Y-%m-%d")
d + timedelta(days=7); d.strftime("%b %d")
(d2 - d1).days; d.weekday()      # Monday = 0
p = Path("data") / "raw" / "a.csv"
p.name, p.stem, p.suffix, p.parent
p.with_suffix(".parquet"); p.exists()
Path("out").mkdir(parents=True, exist_ok=True)
with open(a) as src, open(b, "w") as dst: ...

```

```

from contextlib import contextmanager
@contextmanager
def timer():
    t = time.perf_counter()
    try: yield          # with-body runs here
    finally: print(time.perf_counter() - t)

```

Use with open(...); csv.DictReader for CSV; pd.read_csv for real work.

CONCURRENCY AND MEMORY

- GIL: one thread runs Python bytecode at a time. Threads help I/O-bound work; use multiprocessing / ProcessPoolExecutor for CPU-bound; asyncio for many network waits.
- Memory: reference counting plus a cycle collector. Generators and iterators avoid building big lists.
- Iterator protocol: __iter__ returns self, __next__ raises StopIteration.
- Profiling: timeit, cProfile; vectorise with NumPy before optimising loops.

GOTCHAS

TRAP sum of floats: 0.1 + 0.2 ≠ 0.3; use math.isclose. Modifying a list while iterating over it skips items; iterate over a copy. Integer division // floors toward -inf: -7 // 2 = -4. % has the sign of the divisor. dict order is insertion order; set order is arbitrary. is on ints/strings; except Exception hiding bugs; = on floats.

ERROR MESSAGES -> CAUSE

Message	Usual cause
TypeError: unhashable type: 'list'	list used as dict key or set item
KeyError / IndexError	missing key / index out of range
UnboundLocalError	assigned in a function before use, needs nonlocal/global
RecursionError	no base case (default limit ~1000)
ValueError: too many values to unpack	length mismatch in unpacking
AttributeError: 'NoneType'...	a function returned None (e.g. .sort())

STYLE AND TOOLING

- PEP 8, type hints (def f(x: int) → str), docstrings, small pure functions, f-strings.
- venv + requirements.txt; pytest for tests (assert f(2) == 4); black/ruff for formatting.
- if __name__ == "__main__": guards script code.

SAY IT IN 20 SECONDS

- list vs tuple: mutable vs immutable, tuples are hashable and slightly faster.
- Shallow vs deep copy: shared inner objects vs fully independent.
- Generator vs list: lazy and one-pass vs materialised in memory.
- GIL: one bytecode thread at a time; threads for I/O, processes for CPU.
- Decorator: a function that wraps a function to add behaviour.
- "State complexity for every solution."